LAB2

Lab2

- The deadline for lab2 submission is **12th March at 11:59 pm.**
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- The lab scores will be uploaded to the LMS next week.
- Questions about Lab 2 scores will **only** be accepted **during next lab session.**

- File name : {student_id}_p2-1.c, {student_id}_p2-2.c
- Each code will be tested with 5 different inputs files.
- Each input file will be worth 20 points.
- *Make sure that the output format matches the provided example exactly.*

Evaluation criteria

Category	Evaluation	
p2-1	100	
p2-2	100	
Total	200	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version.*

Lab 2

- **Command line arguments**
  : Command-line arguments are inputs passed to a program at execution time through the terminal.

```
$ filename arg1 arg2 arg3
```

<u>int main(int argc, char *argv[])</u>

- **argc (argument count)**: The number of command-line arguments passed to the program, including the program's name.
- **argv (argument vector)**: An array of strings containing the command-line arguments, where argv[0] is the program's name and argv[1] to argv[argc-1] are the additional arguments.

Lab 2

Command line arguments example

```
$ ./p2 input.txt output.txt
```

int main(int argc, char *argv[])

argc=3

./p2	input.txt	output.txt
argv[0]	argv[1]	argv[2]

Lab 2-1. Generating Permutations Using a Recursive Algorithm

- Implement a program in C that generates **all permutations using a recursive algorithm.**
- The program receives a positive integer *n* as a **command-line argument**.
- Construct a set of integers from 1 to *n* based on the input value by using **dynamic memory allocation**.
- Use a recursive function to systematically construct permutations of this set.
- The program should output all *n!* possible permutations of the set *{1, 2, ..., n}*, each on a separate line.

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ gcc -o 2025123456_p2-1 2025123456_p2-1.c
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./2025123456_p2-1 3
The set of permutations is :
[ 1 2 3 ]
[ 1 3 2 ]
[ 2 1 3 ]
[ 2 3 1 ]
[ 3 2 1 ]
[ 3 1 2 ]
```

Lab 2-1. Generating Permutations Using a Recursive Algorithm

- **Your program should:**

1. Read a **positive integer *n*** from the **command-line argument.**
    1. If the number of command-line arguments is not exactly one, print the following error message and terminate the program : **"Error: Please provide a number as a command-line argument."**
    2. If the provided argument is **not a positive integer ($n \leq 0$)**, print the following error message and terminate the program : **"Error: Please provide a positive number."**
2. Create an array containing the integers **from 1 to *n* by using dynamic memory allocation.**
3. Implement a function **'perm'** that recursively generates all permutations of the set.
4. Implement a function **'swap'** that exchanges the values of two integers using pointers.
5. When a complete permutation is generated, print the elements in the following format :
   "The set of permutations is : ₩n [1 2 3] ₩n … "
6. Print each permutation on a **separate line**.
7. The program should generate and print **all *n!* permutations** of the set {1, 2, …, n}.

Lab 2-1. Generating Permutations Using a Recursive Algorithm

EX 1)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./2025123456_p2-1 4
The set of permutations is :
[ 1 2 3 4 ]
[ 1 2 4 3 ]
[ 1 3 2 4 ]
[ 1 3 4 2 ]
[ 1 4 3 2 ]
[ 1 4 2 3 ]
[ 2 1 3 4 ]
[ 2 1 4 3 ]
[ 2 3 1 4 ]
[ 2 3 4 1 ]
[ 2 4 3 1 ]
[ 2 4 1 3 ]
[ 3 2 1 4 ]
[ 3 2 4 1 ]
[ 3 1 2 4 ]
[ 3 1 4 2 ]
[ 3 4 1 2 ]
[ 3 4 2 1 ]
[ 4 2 3 1 ]
[ 4 2 1 3 ]
[ 4 3 2 1 ]
[ 4 3 1 2 ]
[ 4 1 3 2 ]
[ 4 1 2 3 ]
```

EX 2)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./2025123456_p2-1 -3
Error: Please provide a positive number.
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./2025123456_p2-1
Error: Please provide a number as command-line argument.
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./2025123456_p2-1 3 5 9
Error: Please provide a number as command-line argument.
```

Lab 2-2. Handling Song Playlists Using Structures

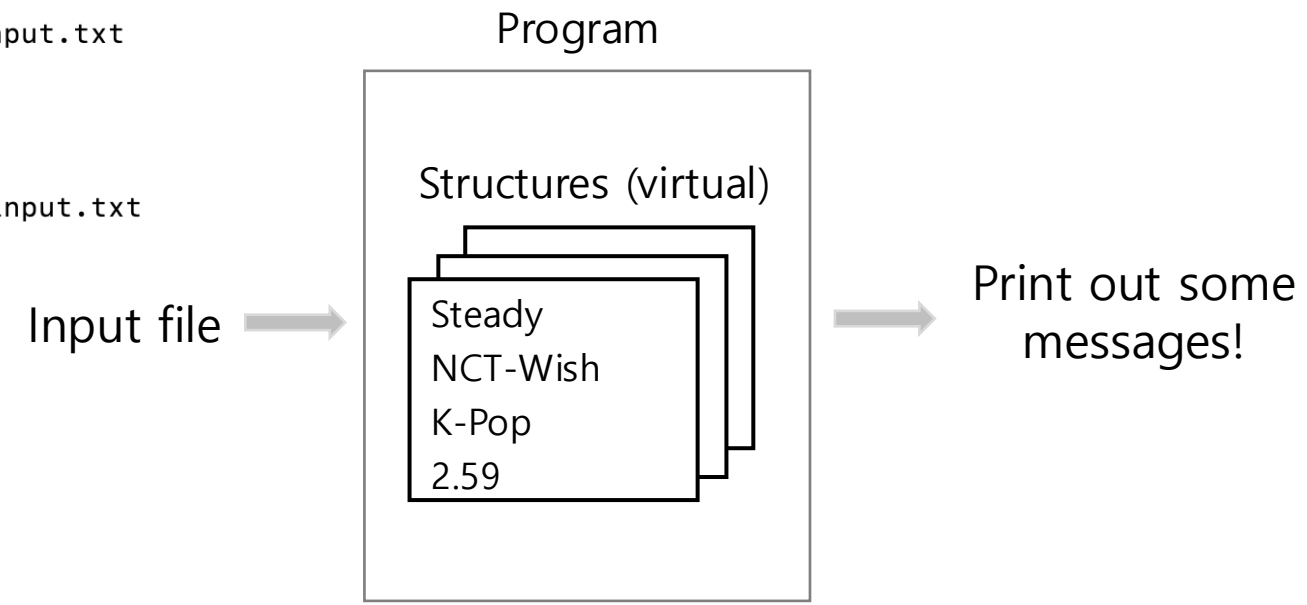
- The input file will contain a list of songs, where each song has the following attributes: Title, Artist, Genre, Duration.
  - The title, artist, and genre will be entered fewer than 30 characters each.
  - The duration will be entered with up to two decimal places.
- Your program should accept the input file as a command-line argument.
- You must use a struct to store this information dynamically.

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ cat input.txt
3
Steady NCT-Wish K-Pop 2.59
SunnyRoad NCT-127 J-Pop 3.21
Boss NCT-U K-Pop 3.30
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./p2 input.txt
Total songs in the playlist: 3

1. Steady by NCT-Wish
   Genre: K-Pop, Duration: 2.59 minutes

2. SunnyRoad by NCT-127
   Genre: J-Pop, Duration: 3.21 minutes

3. Boss by NCT-U
   Genre: K-Pop, Duration: 3.30 minutes
```



Lab 2-2. Handling Song Playlists Using Structures

- **Your program should:**

  1. Check if the input file is provided via the command line. If not, print an error message '**Error: No input file specified. Please provide the input file via a command-line argument.**' and exit.
  2. Open and read the input file.
  3. Read the number of songs from the file.
  4. Dynamically allocate memory for an array of structs.
  5. Read and store each song's details.
  6. Print the list of songs in a formatted way.
  7. Free all allocated memory before exiting.

Lab 2-2. Handling Song Playlists Using Structures

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct song {
    char *title;
    char *artist;
    char *genre;
    float duration;
} song; //creating data structure!!!

int main(int argc, char **args) {
    char *input_path;
    FILE *fi;

    int num, i;

    song *playlist;

        ⋮
```

Lab 2-2. Handling Song Playlists Using Structures

EX 1)

```
[~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ cat input2.txt
4
Dream BTS K-Pop 3.45
LoveAgain Twice J-Pop 2.50
Butter Blackpink K-Pop 2.45
BoyWithLuv Halsey&BTS Pop 3.53
[~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./p2 input2.txt
Total songs in the playlist: 4

1. Dream by BTS
   Genre: K-Pop, Duration: 3.45 minutes

2. LoveAgain by Twice
   Genre: J-Pop, Duration: 2.50 minutes

3. Butter by Blackpink
   Genre: K-Pop, Duration: 2.45 minutes

4. BoyWithLuv by Halsey&BTS
   Genre: Pop, Duration: 3.53 minutes
```

EX 2)

```
[~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab2/code ❯ ./p2
Error: No input file specified. Please provide the input file via a command-line argument.
```